

Running in Production with Docker and Fly.io

Contents

1. Purpose and scope	4
2. The three runtime requirements	5
3. Part 1 — Preventing the missing <code>AGE_KEY</code> error	6
3.1. Injecting the key at container start	6
3.2. Using an untracked env file for local development	6
3.3. Configuring the key as a Fly.io secret	8
4. Part 2 — Preventing the database <code>localhost</code> error	9
5. Part 3 — Preventing the fail-closed boot on security configuration	10
5.1. What the boot log says when it fails	10
5.2. What to set	10
5.3. Where the values come from	11
6. Part 4 — Preventing the trusted-proxy misconfiguration	12
6.1. What the boot log says	12
6.2. Multiple proxies	13
7. Part 5 — Docker Compose	14
7.1. The <code>compose.yaml</code> file	14
7.2. Why each field is there	15
7.3. Running it	16
7.4. Verifying the setup	16
8. Part 6 — Fly.io deployment	18
8.1. One-time setup	18

8.2. What goes in <code>fly.toml</code>	18
8.3. What must not go in <code>fly.toml</code>	20
8.4. Verifying the deployment	20
8.5. Rotating <code>AGE_KEY</code>	21
9. Part 7 — Keeping production values out of local files	22
10. Part 8 — Troubleshooting	23
10.1. <code>no age key available</code>	23
10.2. <code>failed to decrypt secrets.enc.yaml: no identity matched any of the recipients</code>	23
10.3. <code>insecure security-headers configuration: HSTS_MAX_AGE is 0</code>	23
10.4. <code>insecure security-headers configuration: CSP_REPORT_ONLY is true</code>	23
10.5. <code>connection refused or dial tcp ... connect: connection refused on the database</code>	24
10.6. <code>password authentication failed for user "..."</code>	24
10.7. Port 8080 is in use	24
10.8. The Compose healthcheck never reports healthy	25
10.9. <code>pull access denied for mywebapp on docker compose up</code>	25
10.10. Users are being rate-limited after a few requests	25
10.11. <code>docker compose up</code> cannot find <code>host.docker.internal</code>	26
11. Summary	27
12. Files referenced by this document	28

patrick toca v0.8.4, October 10th, 2026

Creation Date: October 7th, 2026

Author: patrick toca

Email: pmjtoca@gmail.com

1. Purpose and scope

This document describes how to run the MyWebApp container in production and in local environments, focusing on the runtime configuration the image needs and the failures that occur when it is missing.

The container image is self-contained. Every failure observed so far has been caused by configuration that must be supplied at **runtime**, not at build time. Three failures dominate:

1. A missing `AGE_KEY`, which the application requires to decrypt its secrets file at boot. The process exits immediately when it is absent.
2. A `DB_HOST` value of `localhost`, which inside a container refers to the container itself, not the host running PostgreSQL.
3. A missing or unsafe security-headers configuration. Since v0.8.4, the boot fails in production and staging when HSTS is off or CSP is report-only.

All three are addressed in this document. The Compose setup described here has been verified end-to-end: the image builds locally, the app connects to PostgreSQL, port 8080 is published, and `/health` returns `{"status":"ok"}`.

By the end of this document you will be able to:

1. Run the container locally against a PostgreSQL instance on the host.
2. Use an untracked env file to keep runtime secrets out of the image and out of git.
3. Bring the same setup up with `docker compose`.
4. Deploy the same image to Fly.io with `AGE_KEY` stored as a Fly Secret.
5. Configure the trusted-proxy list so IP-based rate limiting works behind a load balancer.
6. Diagnose the three common runtime failures from the container logs.

2. The three runtime requirements

The application will not start without these:

1. **AGE_KEY** — the age private key used by `internal/credentials` to decrypt `secrets.enc.yaml`. Without it, the process exits with a clear error before any service is constructed.
2. **A reachable PostgreSQL instance** — the connection is established at startup. The host must be set correctly for the container's network namespace.
3. **A safe security-headers configuration** — in production and staging, `HSTS_MAX_AGE` must be non-zero and `CSP_REPORT_ONLY` must be false. Development tolerates both being at their defaults.

Everything else — port, log format, cookie flags — has a safe default and can be overridden if needed.

3. Part 1 — Preventing the missing `AGE_KEY` error

The application fails fast when `AGE_KEY` is absent. This is intentional: a missing decryption key must never be silently ignored. A server that starts without it is a server that will fail later, under load, in a harder-to-diagnose way.

3.1. Injecting the key at container start

Pass the key directly when starting the container. Read it from your local age key file, stripping comment lines:

```
docker run --rm \
  -e AGE_KEY="$(grep -v '^#' ~/.config/sops/age/keys.txt)" \
  ...
mywebapp:debug
```



Passing secrets on the command line exposes them to shell history and to any user who can run `ps`. This form is acceptable only for a quick local test. Use an env file or a platform secret for anything persistent.

3.2. Using an untracked env file for local development

Create a local environment file containing the required runtime values:

```
APP_ENV=production
AGE_KEY=AGE-SECRET-KEY-...
DB_HOST=host.docker.internal
DB_PORT=5432
DB_USER=pmjtoca
DB_NAME=MyWebApp
DB_SSLMODE=disable
PORT=8080
LOG_FORMAT=text
```

```
LOG_LEVEL=debug
HSTS_MAX_AGE=31536000
HSTS_INCLUDE_SUBDOMAINS=true
CSP_REPORT_ONLY=false
COOKIE_SECURE=false
TRUSTED_PROXIES=
```

Then start the container with that file:

```
docker run --rm \
  --env-file .env.docker.local \
  --add-host host.docker.internal:host-gateway \
  -p 8080:8080 \
  mywebapp:debug
```

`PORT=8080` is set explicitly in the file. The image's default port is already 8080, but setting it in the env file makes the Compose port mapping and the application's listen port agree without relying on the default.

`COOKIE_SECURE=false` is the one production value you cannot honor on plain HTTP. The browser will refuse to store a Secure cookie over <http://localhost>, and login will appear to succeed but every subsequent request will be unauthenticated. Set it to `true` once the container is behind HTTPS.

Add the env file to `.gitignore` so it is never committed:

```
.env.docker.local
.env.docker.local.*
```



Never put `AGE_KEY` in the Dockerfile, in image layers, in Git, or in a committed `.env` file. Any of those locations turns a runtime secret into a permanent, widely distributed artifact.

3.3. Configuring the key as a Fly.io secret

On Fly.io, store the key as a platform secret rather than an environment file:

```
fly secrets set AGE_KEY="$(grep -v '^#' ~/.config/sops/age/keys.txt)" --app mywebapp
```

Fly injects the secret into the running machine at start time, so it never appears in the image or in a repository.

4. Part 2 — Preventing the database `localhost` error

Inside a container, `localhost` refers to the container's own network namespace. PostgreSQL is almost never there. Set `DB_HOST` according to where the database actually runs:

Where PostgreSQL runs	Value for <code>DB_HOST</code>
On the host machine	<code>host.docker.internal</code>
In Docker Compose	The database service name, for example <code>db</code>
On Fly.io	The private or managed database hostname
Anywhere else	Its reachable DNS name or IP address

Your local `docker run` command should therefore include:

```
-e DB_HOST=host.docker.internal
```

On Linux, `host.docker.internal` does not resolve by default. Add the mapping explicitly:

```
--add-host host.docker.internal:host-gateway
```

On Docker Desktop for macOS and Windows, the mapping is provided automatically; the `--add-host` flag is harmless but unnecessary there.

In addition to setting `DB_HOST`, ensure that PostgreSQL itself accepts connections from Docker:

- Its `listen_addresses` setting must include the interface the container reaches it on. For a PostgreSQL listening only on `localhost`, set it to `'*'` or to the Docker bridge address.
- Its `pg_hba.conf` must contain a rule permitting the Docker network or the container's source address, with `md5`, `scram-sha-256`, or `trust` as appropriate.
- Any host firewall must allow traffic from the Docker bridge network to the PostgreSQL port.

5. Part 3 — Preventing the fail-closed boot on security configuration

Since v0.8.4, the application refuses to start in production and staging when either of the following is true:

- `HSTS_MAX_AGE` is `0`. A deployed environment that does not send `Strict-Transport-Security` permits a protocol downgrade.
- `CSP_REPORT_ONLY` is `true`. A deployed environment with a report-only CSP has no CSP enforcement.

The check is in `LoadSecurityHeadersConfig` and fires after `credentials.Load` and before the database pool is opened. When it fires, the process exits with a message naming the offending variable and the environment it saw.

5.1. What the boot log says when it fails

```
{"level":"ERROR","msg":"security headers configuration failed",  
  "error":"insecure security-headers configuration:  
  HSTS_MAX_AGE is 0 in production; set it to at least 31536000  
  (APP_ENV=production)"}
```

The process exits with code 1. Nothing listens on the port.

5.2. What to set

For a deployed environment, set:

```
APP_ENV=production  
HSTS_MAX_AGE=31536000  
HSTS_INCLUDE_SUBDOMAINS=true  
HSTS_PRELOAD=false
```

```
CSP_REPORT_ONLY=false
```

`HSTS_PRELOAD=false` is the safe default. Preload commits the domain to HTTPS permanently across a shipped list of browsers; submission is not reversible in under six weeks. Leave it off until the domain has been stable for months.

`CSP_REPORT_ONLY=false` turns the policy from "report violations" to "enforce violations". Confirm in a staging environment that no page breaks before flipping this in production.

5.3. Where the values come from

In a container run, the values come from the `ENV` block in the `Dockerfile` or from an `--env-file`. The v0.8.4 image ships the production values in its `ENV` block, so a plain `docker run` boots with the safe configuration.

In a Fly deployment, the values come from `fly.toml`'s `[env]` block. The next section shows the block.

6. Part 4 — Preventing the trusted-proxy misconfiguration

The rate limiter keys on the client IP returned by Gin's `ClientIP()`. That function reads `X-Forwarded-For` only when the immediate peer of the connection is on a trusted network. The trusted network comes from the `TRUSTED_PROXIES` environment variable, a comma-separated list of CIDRs.

The default is empty, which means "trust nothing". That is correct for a direct `docker run` from the host. On Fly, the load balancer is on `fdaa::/16`, and that value belongs in `fly.toml`:

```
[env]
  TRUSTED_PROXIES = "fdaa::/16"
```

Without it, every request looks like it comes from the load balancer and the rate limiter treats the entire user base as a single client. The symptom is that legitimate users are throttled after a handful of requests, even though each of them makes only one.

6.1. What the boot log says

On a successful boot, the log contains one of:

```
INFO trusted proxies configured cidrs=[fdaa::/16]
```

or, when the list is empty:

```
INFO trusted proxies: none (X-Forwarded-For is ignored)
```

Read this line after every deploy. If it shows `none` and you expect `fdaa::/16`, the variable is not reaching the process.

6.2. Multiple proxies

If your deployment has more than one proxy in front of the application — Cloudflare, then Fly, then something else — list every proxy in the chain, comma-separated, from the outermost inward. The value is the union of every network that can legitimately place a value in `X-Forwarded-For`. If you are unsure, leave the list empty. The failure mode of an empty list is "the rate limiter keys on the load balancer", which is imprecise but not exploitable. The failure mode of an over-broad list is "any client in the trusted range can spoof the header", which is exploitable.

7. Part 5 — Docker Compose

For local development, replace ad-hoc `docker run` invocations with a Compose file. This makes the runtime configuration explicit and reproducible, and it is what the repository ships with.

7.1. The `compose.yaml` file

```
services:
  app:
    build:
      context: .
      dockerfile: Dockerfile
    image: mywebapp:local
    pull_policy: never          # never try to pull; build locally
    ports:
      - "8080:8080"
    env_file:
      - .env.docker.local
    extra_hosts:
      - "host.docker.internal:host-gateway"
    networks:
      - mywebapp-net
    healthcheck:
      test: ["CMD", "wget", "-q0-", "http://localhost:8080/health"]
      interval: 30s
      timeout: 5s
      retries: 3
      start_period: 10s
      restart: unless-stopped

networks:
  mywebapp-net:
```

```
driver: bridge
```

7.2. Why each field is there

`build.context` and `dockerfile`

Build the image from the repository root using the project's `Dockerfile`. There is no registry involved.

`image: mywebapp:local` and `pull_policy: never`

Tag the locally built image and instruct Compose not to attempt a registry pull.

`ports`

Publish the application's listen port on the host. The left side is the host port; the right side is the container port. Both are 8080 because `.env.docker.local` sets `PORT=8080`.

`env_file`

Read `.env.docker.local` and pass each key as an environment variable into the container. This is where `AGE_KEY`, `DB_HOST`, and the security-headers values live.

`extra_hosts`

Map `host.docker.internal` to the host gateway inside the container. On Linux, Docker does not provide this name by default.

`networks`

Attach the container to a named bridge network. The named network behaves predictably.

`healthcheck.test`

A `wget` against `/health`. The endpoint does no I/O, so the healthcheck is a cheap liveness signal. `wget -q0-` is a GET request.

`restart: unless-stopped`

Restart the container if it exits, except when it was explicitly stopped by the user.

7.3. Running it

```
docker compose up --build
```

To run in the background:

```
docker compose up -d
```

To stop and remove the container:

```
docker compose down
```

7.4. Verifying the setup

Four checks apply to any new environment.

7.4.1. 1. The image builds locally

```
docker compose build
```

Expected: a successful build, ending with a **Successfully tagged mywebapp:local** line.

7.4.2. 2. The app connects to PostgreSQL

```
docker compose logs app | grep -i "pool\|database"
```

Expected: a log line indicating the pool was configured.

7.4.3. 3. Port 8080 is published

```
docker compose ps
```

Expected: `0.0.0.0:8080→8080/tcp` in the `PORTS` column.

7.4.4. 4. `/health` returns `{"status":"ok"}`

```
curl -s http://localhost:8080/health
```

Expected: `{"status":"ok"}`.

7.4.5. 5. The boot log names the security configuration

```
docker compose logs app | grep -E 'trusted proxies|security headers'
```

Expected:

```
INFO trusted proxies: none (X-Forwarded-For is ignored)  
INFO security headers configuration hsts_max_age=31536000 hsts_include_subdomains=true hsts_preload=false csp_report_only=false
```

If the second line is missing, the boot failed before reaching it. Read the error above it.

8. Part 6 — Fly.io deployment

Fly.io runs the same image. The only difference is where the runtime configuration comes from: an env file locally, Fly Secrets and `fly.toml`'s `[env]` block in production.

8.1. One-time setup

```
fly auth login
fly launch --no-deploy      # generates/updates fly.toml, if needed
fly secrets set AGE_KEY="$(grep -v '^#' ~/.config/sops/age/keys.txt)"
fly deploy
```

`AGE_KEY` is the only Fly Secret that needs to be set. Every other secret — `JWT_SECRET`, `CSRF_SECRET`, `DB_PASSWORD`, the R2 credentials — lives in `secrets.enc.yaml`, which is decrypted at boot by `internal/credentials`. Storing them again as Fly Secrets would create a second source of truth and defeat the design. See [the age/SOPS/Fly how-to](#) for the full model.

8.2. What goes in `fly.toml`

Non-secret configuration. The `[env]` block is visible in the Fly dashboard and in the file itself, so it holds only values that are not sensitive:

```
[env]
  APP_ENV = "production"
  PORT = "8080"
  LOG_FORMAT = "json"
  LOG_LEVEL = "info"
  GIN_MODE = "release"

  # Security headers – required by the fail-closed check in
  # LoadSecurityHeadersConfig. Without these, the boot aborts with
  # "insecure security-headers configuration".
```

```
HSTS_MAX_AGE = "31536000"
HSTS_INCLUDE_SUBDOMAINS = "true"
HSTS_PRELOAD = "false"
CSP_REPORT_ONLY = "false"

# Cookies
COOKIE_SECURE = "true"
COOKIE_SAMESITE = "lax"

# Trusted proxies – Fly's internal network.
TRUSTED_PROXIES = "fd::/16"

# Database coordinates. The password is in secrets.enc.yaml.
DB_HOST = "mywebapp-db.internal"
DB_PORT = "5432"
DB_USER = "mywebapp"
DB_NAME = "mywebapp"
DB_SSLMODE = "require"

[http_service]
  internal_port = 8080
  force_https = true
  auto_stop_machines = "stop"
  auto_start_machines = true
  min_machines_running = 0

[[http_service.checks]]
  interval = "30s"
  timeout = "5s"
  grace_period = "10s"
  method = "GET"
  path = "/health"
```

`internal_port` matches the `PORT` the application listens on. The `[[http_service.checks]]` block is the Fly equivalent of the Compose healthcheck; it hits `/health` with a

GET.

8.3. What must not go in `fly.toml`

- `AGE_KEY` — a secret, belongs in `fly secrets`.
- `JWT_SECRET`, `CSRF_SECRET` — same.
- `DB_PASSWORD` — same.
- Any R2 credential — same.

If a value in `[env]` would be redacted in a log line, it belongs in `fly secrets`, not in `[env]`.

8.4. Verifying the deployment

```
fly status
fly logs | grep -E 'trusted proxies|security headers'
curl -s https://mywebapp.fly.dev/health | jq .
curl -s https://mywebapp.fly.dev/version | jq .
```

Expected:

- `status` is `ok`.
- The log shows `trusted proxies configured cidrs=[fdaa::/16]` and `security headers configuration hsts_max_age=31536000 ... csp_report_only=false`.
- `/health` returns `{"status":"ok"}`.
- `/version` reports the tag and commit that were deployed.

If either log line is missing, the boot failed. Read the error above it; it names the variable to fix.

8.5. Rotating AGE_KEY

See the "Rotating the age key" section of [the age/SOPS/Fly how-to](#). The short version:

```
# 1. Generate a new key, re-encrypt secrets.enc.yaml to it, keep the old
#    recipient as a fallback.
# 2. Set the new key on Fly.
fly secrets set AGE_KEY="$(cat ~/.config/sops/age/keys-new.txt)"

# 3. Deploy and confirm healthy.
fly deploy
fly logs

# 4. Remove the old recipient from secrets.enc.yaml, commit, deploy.
```

The re-encryption must happen before the old key is discarded. There is no way to decrypt a file with a key it was not encrypted to.

9. Part 7 — Keeping production values out of local files

Local env files are a convenience for development only. For production:

- Keep every secret in Fly secrets, or your platform's equivalent secret store.
- Keep non-secret configuration in the deployment configuration, not in a committed env file.
- Keep the application's fail-fast behavior. The process should continue to exit immediately when the age key, the database configuration, or the security-headers configuration is missing or unsafe.

The `.gitignore` entries that support this:

```
.env
.env.*
!.env.example
.env.docker.local
.env.docker.local.*
secrets.yaml
secrets.yml
secrets.env
```

`secrets/secrets.enc.yaml` is deliberately **not** ignored. It is the ciphertext, it is safe to commit, and `git diff` on it is meaningful because SOPS encrypts the values but leaves the keys readable.

10. Part 8 — Troubleshooting

10.1. no age key available

The error from `credentials.load`. Causes, in order of likelihood:

1. `AGE_KEY` is unset. Locally, check `.env.docker.local`. On Fly, check `fly secrets list`.
2. The env file was not read. Confirm the `--env-file` flag or the `env_file:` entry in `compose.yaml`.
3. The key file at `~/.config/sops/age/keys.txt` is missing or has permissions looser than `0600`.

10.2. failed to decrypt secrets.enc.yaml: no identity matched any of the recipients

The `AGE_KEY` supplied to the container is not the one the file was encrypted to. Either the wrong key was passed, or the file was re-encrypted after the key was copied. Retrieve the correct key or re-encrypt the file to the current recipient.

10.3. insecure security-headers configuration: HSTS_MAX_AGE is 0

The fail-closed check fired. Either set `HSTS_MAX_AGE` to a non-zero value (`31536000` is standard), or set `APP_ENV=development` if you are running the container locally and do not want the production posture.

10.4. insecure security-headers configuration: CSP_REPORT_ONLY is true

Same check, different variable. Either set `CSP_REPORT_ONLY=false`, or set `APP_ENV=development`.

10.5. connection refused or dial tcp ... connect: connection refused on the database

The container reached the host but PostgreSQL is not listening on the address it tried. Common causes:

1. `DB_HOST` is `localhost` and the container is talking to itself. Set it to `host.docker.internal` and pass `--add-host host.docker.internal:host-gateway`.
2. PostgreSQL is listening only on `127.0.0.1`. Set `listen_addresses` to `'*'` or to the Docker bridge address.
3. `pg_hba.conf` has no rule for the container's source address.
4. A host firewall is blocking traffic from the Docker bridge.

10.6. password authentication failed for user "..."

The password in `DB_PASSWORD` (loaded from `secrets.enc.yaml`) does not match the password for the role in PostgreSQL. Either update the encrypted file, or change the role's password.

10.7. Port 8080 is in use

Another process holds the port. Find it:

```
sudo ss -ltnp | grep :8080
```

Either stop the other process, or change the host-side port mapping in `compose.yaml`:

```
ports:  
  - "8081:8080"
```

The container still listens on 8080; only the host port changes.

10.8. The Compose healthcheck never reports healthy

Check the log:

```
docker compose logs app
```

If the application is logging normally, the healthcheck itself is failing. Two common causes:

1. `wget` is not present in the image. The healthcheck depends on it.
2. The healthcheck runs before the database pool is ready and the retry count is exhausted. Increase `start_period` and `retries`.

10.9. pull access denied for mywebapp on docker compose up

Compose is trying to pull the image from a registry instead of using the locally built one. Confirm that `compose.yaml` has both:

```
image: mywebapp:local  
pull_policy: never
```

and that the build succeeds with `docker compose build` before `docker compose up`.

10.10. Users are being rate-limited after a few requests

The trusted-proxy list is probably empty on a deployment that has a load balancer. Check the boot log:

```
fly logs | grep 'trusted proxies'
```

If it shows `none` and the deployment is behind a proxy, set `TRUSTED_PROXIES` in `fly.toml` to the load balancer's CIDR. On Fly, that value is `fdaa::/16`.

10.11. docker compose up cannot find host.docker.internal

On Linux, this name is not provided by default. Confirm that the service has:

```
extra_hosts:  
  - "host.docker.internal:host-gateway"
```

and that Docker is recent enough to support the `host-gateway` special value (Docker 20.10 or later).

11. Summary

1. Supply `AGE_KEY` at runtime, through an untracked env file locally or a platform secret in production.
2. Set `DB_HOST` to the real location of PostgreSQL, never to `localhost` from inside a container.
3. Set `HSTS_MAX_AGE` and `CSP_REPORT_ONLY` to their production values. The boot refuses to start otherwise.
4. Set `TRUSTED_PROXIES` to the load balancer's CIDR. On Fly, that is `fdaa::/16`. The default is empty (trust nothing).
5. Use `compose.yaml` for local runs, with `pull_policy: never` to prevent registry pulls, a named bridge network, and `host.docker.internal:host-gateway` for host access.
6. Use a GET-compatible healthcheck against `/health`.
7. Use Fly secrets for `AGE_KEY` and nothing else; everything else lives in `secrets.enc.yaml`.
8. Preserve fail-fast behavior on missing or unsafe configuration.

12. Files referenced by this document

Path	Purpose
Dockerfile	The image build definition. Since v0.8.4 its <code>ENV</code> block carries the production security-headers values.
compose.yaml	The local Compose service definition.
.env.docker.local	The untracked local env file. Gitignored. Contains <code>AGE_KEY</code> , <code>DB_HOST</code> , the security-headers values, and the other runtime variables.
fly.toml	The Fly application manifest. Contains non-secret configuration only, including <code>TRUSTED_PROXIES</code> and the security-headers values.
secrets/secrets.enc.yaml	The committed, SOPS-encrypted secrets file. Decrypted at boot by <code>internal/credentials</code> .
internal/credentials/credentials.go	The loader that reads <code>AGE_KEY</code> and exports the decrypted secrets into the environment.
internal/middleware/security_headers.go	The middleware that emits the headers and the loader that enforces the fail-closed check.
internal/middleware/proxies.go	The loader that reads <code>TRUSTED_PROXIES</code> .
AGE_KEY	The Fly Secret holding the age private key.